

PySpark Project — Coffee Shop Sales

Bronze → Silver → Gold | Joins, Data Cleaning, CTEs & Window Functions

PySpark

SparkSQL

Joins

Data Cleaning

CTEs

Window Functions

PySpark Project — Coffee Shop Sales (Bronze → Silver → Gold)

A hands-on **PySpark project** using a synthetic, relational retail dataset built specifically for practice (a coffee-shop style sales dataset). Unlike a single flat report table, this dataset has **real tables with foreign keys**, so we get to practice actual **joins** on top of everything else.

The raw ("bronze") files

- `customers.csv` — customers (id, name, region, segment, signup date)
- `products.csv` — products (id, name, category, subcategory, cost, price)
- `orders_raw.csv` — orders, + raw order lines, **intentionally a bit messy**: a few duplicate rows, a few missing `customer_id` / `payment_method` values, and a few `quantity` values — just like real raw data.

The project follows a medallion architecture, mirroring your SQL project

- **Bronze** — load the raw CSVs as-is, inspect them, find the data quality issues
- **Silver** — clean + join the tables into one trustworthy, analysis-ready table
- **Gold** — build aggregated business reports (by category, region, month) using `group by`, SparkSQL **CTEs**, and **window functions**

Every code cell is commented **line by line**.

. Start Spark

In [1]:

```
# %pip install pyspark # uncomment if pyspark isn't installed yet

# SparkSession is the entry point for DataFrame & SQL operations
from pyspark.sql import SparkSession

# Import SQL functions under the alias "F" (col, sum, avg, rank, lag, etc.)
from pyspark.sql import functions as F

# Import Window, needed to define window specifications for ranking/running totals
from pyspark.sql.window import Window

# Build (or reuse) a local SparkSession running on all available CPU cores
spark = (
    SparkSession.builder
    .master("local[*]")
    .appName("PySpark_Project_CoffeeShop")
    .getOrCreate()
)

# Keep the console output clean: only show warnings and errors
spark.sparkContext.setLogLevel("WARN")

print("Spark version:", spark.version)
```

```
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-
defaults.properties
26/07/19 15:58:13 WARN Utils: Your hostname, vm, resolves to a loopback
address: 127.0.0.1; using 192.0.2.2 instead (on interface eth0)
26/07/19 15:58:13 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to
another address
Using Spark's default log4j profile: org/apache/spark/log4j2-
defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use
setLogLevel(newLevel).
/usr/local/lib/python3.12/dist-packages/pyspark/testing/utlis.py:127:
FutureWarning: PySpark does not yet fully support pandas >= 3.0.0. Some
features may not work correctly. It is recommended to use pandas < 3.0.0
now.
    require_minimum_pandas_version()
26/07/19 15:58:17 WARN NativeCodeLoader: Unable to load native-hadoop lib
for your platform... using builtin-java classes where applicable
Spark version: 4.2.0
```

. Bronze layer — load the raw files exactly as they are

The bronze layer is a straight, unmodified load of the source files. We inspect them here to identify the data quality problems before fixing anything.

In [2]:

```
# Read each raw CSV into its own DataFrame
# header=True -> first row holds the column names
# inferSchema=True -> Spark guesses each column's data type from the values
bronze_customers = spark.read.csv("customers.csv", header=True, inferSchema=True)
bronze_products = spark.read.csv("products.csv", header=True, inferSchema=True)
bronze_orders = spark.read.csv("orders_raw.csv", header=True, inferSchema=True)

# Quick row counts for all three bronze tables
print("bronze_customers:", bronze_customers.count(), "rows")
print("bronze_products :", bronze_products.count(), "rows")
print("bronze_orders  :", bronze_orders.count(), "rows")
```

[Stage 0:>
1]

(0 +

```
bronze_customers: 300 rows
bronze_products : 24 rows
bronze_orders   : 6025 rows
```

In [3]:

```
# Look at the orders schema and a sample of rows
bronze_orders.printSchema()
bronze_orders.show(5)
```

```
root
|-- order_id: integer (nullable = true)
|-- order_date: date (nullable = true)
|-- customer_id: integer (nullable = true)
|-- product_id: integer (nullable = true)
|-- quantity: integer (nullable = true)
|-- discount_pct: integer (nullable = true)
|-- payment_method: string (nullable = true)
|-- channel: string (nullable = true)

+-----+-----+-----+-----+-----+-----+
+-----+-----+
|order_id|order_date|customer_id|product_id|quantity|discount_pct|
payment_method|   channel|
+-----+-----+-----+-----+-----+-----+
+-----+-----+
|    1617|2024-08-10|    137|      3|      3|      0|
Card|    Online|
|    2125|2024-06-02|    279|      3|      4|     10| Mobile
Wallet|    Online|
|    3915|2024-12-21|    196|      7|      3|      0| Mobile
Wallet|Mobile App|
|    1381|2024-04-14|     95|      4|      1|     10| Mobile
Wallet| In-Store|
|    2806|2024-07-24|     46|      7|      1|      0| Mobile
Wallet|    Online|
+-----+-----+-----+-----+-----+-----+
+-----+-----+
only showing top 5 rows
```

In [4]:

```
# --- Data quality check 1: duplicate rows ---
# dropDuplicates() with no arguments removes fully-identical duplicate rows;
# comparing that count to the original count tells us how many dupes exist
distinct_count = bronze_orders.dropDuplicates().count()
total_count = bronze_orders.count()
print("Total rows:", total_count)
print("Distinct rows:", distinct_count)
print("Duplicate rows:", total_count - distinct_count)
```

[Stage 16:>
1]

(0 +

Total rows: 6025
Distinct rows: 6000
Duplicate rows: 25

In [5]:

```
# --- Data quality check 2: nulls in key columns ---
# For each column, count how many rows have a NULL value in it
bronze_orders.select(
    [F.count(F.when(F.col(c).isNull(), c)).alias(c) for c in bronze_orders.columns]
).show()
```

```
+-----+-----+-----+-----+-----+-----+
+-----+-----+
|order_id|order_date|customer_id|product_id|quantity|discount_pct|
payment_method|channel|
+-----+-----+-----+-----+-----+-----+
+-----+-----+
|      0|      0|      86|      0|      0|      0|
63|      0|
+-----+-----+-----+-----+-----+-----+
+-----+-----+
```

In [6]:

```
# --- Data quality check 3: invalid (negative) quantities ---
# filter() keeps only rows where quantity is less than 0 -- these are bad records
bronze_orders.filter(F.col("quantity") < 0).show(5)
```

```
+-----+-----+-----+-----+-----+-----+
+-----+-----+
|order_id|order_date|customer_id|product_id|quantity|discount_pct|
payment_method|channel|
+-----+-----+-----+-----+-----+-----+
+-----+-----+
|   5820|2024-05-30|      135|      7|      -1|      15|
Card|Mobile App|
|   1138|2024-02-16|      159|      5|      -3|      0|
Cash|Online|
|   1401|2024-04-25|      233|      5|      -3|      15|
Cash|Mobile App|
|   2204|2024-08-13|      82|     19|      -1|      0|
Cash|Mobile App|
|    849|2024-06-30|      55|      1|      -1|      5|
Card|Online|
+-----+-----+-----+-----+-----+-----+
+-----+-----+
only showing top 5 rows
```

. Silver layer — clean the data and join into one

Rules applied here:

- . Drop exact duplicate rows
- . Drop rows with a NULL `customer_id` or `payment_method` (can't analyze what to identify)
- . Keep only rows where `quantity > 0`
- . Join in customer and product details
- . Calculate a proper `line_total` ($\text{quantity} \times \text{price} \times (1 - \text{discount})$)

In [7]:

```
# Step 1: remove exact duplicate rows
orders_dedup = bronze_orders.dropDuplicates()

# Step 2: drop rows missing a customer_id or payment_method
orders_clean = orders_dedup.dropna(subset=["customer_id", "payment_method"])

# Step 3: keep only valid (positive) quantities
orders_clean = orders_clean.filter(F.col("quantity") > 0)

# Confirm how many rows survived the cleaning
print("Rows after cleaning:", orders_clean.count(), "/ original", bronze_orders.count())
```

Rows after cleaning: 5796 / original 6025

In [8]:

```
# Step 4: join the cleaned orders to customers (on customer_id)
orders_with_customer = orders_clean.join(
    bronze_customers,
    on="customer_id",      # the shared key between the two tables
    how="inner"            # keep only orders that match a real customer
)

# Step 5: join the result to products (on product_id)
silver_orders = orders_with_customer.join(
    bronze_products,
    on="product_id",
    how="inner"
)

# Step 6: calculate a proper line_total using price, quantity and discount
silver_orders = silver_orders.withColumn(
    "line_total",
    F.round(
        F.col("quantity") * F.col("unit_price") * (1 - F.col("discount_pct") / 100),
        2
    )
)

# Preview the fully joined, cleaned "silver" table
silver_orders.select(
    "order_id", "order_date", "customer_name", "product_name",
    "category", "quantity", "unit_price", "discount_pct", "line_total"
).show(5)
```

[Stage 40:>
1]

(0 +

```
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+
|order_id|order_date|customer_name| product_name| category|quantity|
|unit_price|discount_pct|line_total|
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+
|      2595|2024-09-21|    Heba Zaki|    Cold Brew| Beverages|
1|         60|         0|    60.0|
|      5394|2024-08-09|    Tarek Adly|        Latte| Beverages|
1|         55|         5|    52.25|
|      3760|2024-12-28|  Amina Naguib| Ceramic Mug|Merchandise|      1|
120|          5|    114.0|
|      1664|2024-06-16|  Fady Mostafa|    Cold Brew| Beverages|
1|         60|         0|    60.0|
|      5999|2024-05-13|  Yasmin Aziz|Travel Tumbler|Merchandise|      1|
180|          0|    180.0|
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+
only showing top 5 rows
```

In [9]:

```
# Cache the silver table in memory since we'll query it many times below
silver_orders.cache()

# Register it as a SQL temp view so we can also query it with spark.sql()
silver_orders.createOrReplaceTempView("silver_orders")

print("Silver table ready:", silver_orders.count(), "clean order lines")
```

```
[Stage 47:>                                     (0 + 1)
200]

[Stage 47:=>                                     (5 + 1)
200]

[Stage 47:==>                                   (10 + 1)
200]

[Stage 47:====>                                (15 + 1)
200]

[Stage 47:=====>                             (20 + 1)
200]

[Stage 47:=====>                             (26 + 1)
200]

[Stage 47:=====>                             (32 + 1)
200]

[Stage 47:=====>                             (38 + 1)
200]

[Stage 47:=====>                             (44 + 1)
200]

[Stage 47:=====>                             (51 + 1)
200]

[Stage 47:=====>                             (58 + 1)
200]

[Stage 47:=====>                             (65 + 1)
200]

[Stage 47:=====>                             (72 + 1)
200]

[Stage 47:=====>                             (79 + 1)
200]

[Stage 47:=====>                             (88 + 1)
200]

[Stage 47:=====>                             (96 + 1)
200]

[Stage 47:=====>                             (104 + 1)
200]
```


[Stage 47:=====> 200]	(112 + 1)
[Stage 47:=====> 200]	(120 + 1)
[Stage 47:=====> 200]	(129 + 1)
[Stage 47:=====> 200]	(139 + 1)
[Stage 47:=====> 200]	(148 + 1)
[Stage 47:=====> 200]	(158 + 1)
[Stage 47:=====> 200]	(169 + 1)
[Stage 47:=====> 200]	(179 + 1)
[Stage 47:=====> 200]	(191 + 1)
[Stage 49:=====> 200]	(28 + 1)
[Stage 49:=====> 200]	(44 + 1)
[Stage 49:=====> 200]	(59 + 1)
[Stage 49:=====> 200]	(73 + 1)
[Stage 49:=====> 200]	(91 + 1)
[Stage 49:=====> 200]	(111 + 1)
[Stage 49:=====> 200]	(130 + 1)
[Stage 49:=====> 200]	(149 + 1)
[Stage 49:=====> 200]	(166 + 1)
[Stage 49:=====> 200]	(188 + 1)
Silver table ready: 5796 clean order lines	

. Gold layer — business reports with `groupBy`

In [10]:

```
# Total revenue and order count by product category
category_report = (
    silver_orders.groupBy("category")
    .agg(
        F.countDistinct("order_id").alias("num_orders"), # unique orders touching this category
        F.sum("line_total").alias("total_revenue"),        # total revenue for the category
        F.round(F.avg("line_total"), 2).alias("avg_line_value") # average value per order line
    )
    .orderBy(F.col("total_revenue").desc())
)

category_report.show()
```

```
[Stage 54:===> (12 + 1)
200]

[Stage 54:====> (16 + 1)
200]

[Stage 54:=====> (23 + 1)
200]

[Stage 54:=====> (29 + 1)
200]

[Stage 54:=====> (35 + 1)
200]

[Stage 54:=====> (41 + 1)
200]

[Stage 54:=====> (47 + 2)
200]

[Stage 54:=====> (49 + 1)
200]

[Stage 54:=====> (55 + 2)
200]

[Stage 54:=====> (61 + 1)
200]

[Stage 54:=====> (68 + 1)
200]

[Stage 54:=====> (73 + 1)
200]

[Stage 54:=====> (79 + 1)
200]

[Stage 54:=====> (85 + 1)
200]

[Stage 54:=====> (94 + 1)
200]
```

```

[Stage 54:=====> (102 + 1)
200]

[Stage 54:=====> (111 + 1)
200]

[Stage 54:=====> (118 + 1)
200]

[Stage 54:=====> (128 + 1)
200]

[Stage 54:=====> (137 + 1)
200]

[Stage 54:=====> (147 + 1)
200]

[Stage 54:=====> (156 + 1)
200]

[Stage 54:=====> (163 + 1)
200]

[Stage 54:=====> (171 + 1)
200]

[Stage 54:=====> (181 + 1)
200]

[Stage 54:=====> (190 + 1)
200]

[Stage 54:=====>(198 + 1)
200]

```

```

+-----+-----+-----+-----+
| category|num_orders| total_revenue|avg_line_value|
+-----+-----+-----+-----+
|Merchandise|    947|    557478.5|    588.68|
|  Beverages|   2403|199209.24999999994|    82.9|
|    Snacks|   1182|    130077.2|   110.05|
|    Bakery|   1264|108845.79999999994|    86.11|
+-----+-----+-----+-----+

```

In [11]:

```
# Revenue by region and customer segment together
region_segment_report = (
    silver_orders.groupBy("region", "segment")
    .agg(F.sum("line_total").alias("total_revenue"))
    .orderBy(F.col("total_revenue").desc())
)

region_segment_report.show(15)
```

```
[Stage 63:=====> (33 + 1)
200]

[Stage 63:=====> (44 + 1)
200]

[Stage 63:=====> (54 + 1)
200]

[Stage 63:=====> (63 + 1)
200]

[Stage 63:=====> (72 + 1)
200]

[Stage 63:=====> (85 + 1)
200]

[Stage 63:=====> (94 + 1)
200]

[Stage 63:=====> (107 + 1)
200]

[Stage 63:=====> (119 + 1)
200]

[Stage 63:=====> (132 + 1)
200]

[Stage 63:=====> (144 + 1)
200]

[Stage 63:=====> (157 + 1)
200]

[Stage 63:=====> (166 + 1)
200]

[Stage 63:=====> (175 + 1)
200]

[Stage 63:=====> (184 + 1)
200]

[Stage 63:=====> (194 + 1)
200]
```

```

+-----+-----+-----+
|   region|segment|   total_revenue|
+-----+-----+-----+
|  Mansoura|Regular| 89210.64999999998|
|   Sharqia|Regular| 66553.70000000001|
|Alexandria|   New|      65157.0|
|     Giza|   New|      62632.85|
|     Tanta|   New| 56905.94999999999|
|     Giza|Regular| 54348.69999999999|
|   Sharqia|   New|      51281.75|
|    Luxor|Regular|      46184.95|
|    Cairo|   New|      44216.8|
|    Aswan|   New|43852.950000000004|
|    Cairo|   VIP|43817.49999999985|
|    Tanta|Regular|41526.100000000006|
|  Mansoura|   New|40703.399999999994|
|Alexandria|Regular| 37001.49999999999|
|    Aswan|Regular|35937.14999999994|
+-----+-----+-----+
only showing top 15 rows

```

In [12]:

```
# Monthly sales trend
# date_format() extracts a "YYYY-MM" string from the order_date column
monthly_trend = (
    silver_orders
    .withColumn("order_month", F.date_format(F.col("order_date"), "yyyy-MM"))
    .groupBy("order_month")
    .agg(F.sum("line_total").alias("monthly_revenue"))
    .orderBy("order_month")
)

monthly_trend.show(12)
```

```
[Stage 68:=====> (26 + 1)
200]

[Stage 68:=====> (36 + 1)
200]

[Stage 68:=====> (47 + 1)
200]

[Stage 68:=====> (58 + 1)
200]

[Stage 68:=====> (67 + 1)
200]

[Stage 68:=====> (81 + 1)
200]

[Stage 68:=====> (93 + 1)
200]

[Stage 68:=====> (109 + 1)
200]

[Stage 68:=====> (123 + 1)
200]

[Stage 68:=====> (138 + 1)
200]

[Stage 68:=====> (153 + 1)
200]

[Stage 68:=====> (165 + 1)
200]

[Stage 68:=====> (178 + 1)
200]

[Stage 68:=====> (189 + 1)
200]
```

```

+-----+-----+
|order_month|  monthly_revenue|
+-----+-----+
|    2024-01|          74936.45|
|    2024-02| 82181.59999999999|
|    2024-03| 75586.04999999997|
|    2024-04| 83101.80000000002|
|    2024-05| 90656.14999999998|
|    2024-06| 76315.55000000002|
|    2024-07|           89266.0|
|    2024-08|106032.79999999999|
|    2024-09|          68618.25|
|    2024-10| 91689.79999999999|
|    2024-11| 72579.94999999998|
|    2024-12| 84646.34999999998|
+-----+-----+

```

. SparkSQL with a CTE — top customers
region

In [13]:

```
# WITH ... AS (...) defines a temporary named result we can reuse in the main query
top_customers_query = spark.sql("""
    WITH customer_totals AS (
        -- Step 1: total revenue per customer, per region
        SELECT region,
               customer_name,
               SUM(line_total) AS customer_revenue
        FROM silver_orders
        GROUP BY region, customer_name
    ),
    ranked_customers AS (
        -- Step 2: rank customers within each region by revenue
        SELECT *,
               RANK() OVER (PARTITION BY region ORDER BY customer_revenue DESC) AS region_rank
        FROM customer_totals
    )
    -- Step 3: keep only the top 5 customers per region
    SELECT region, customer_name, customer_revenue, region_rank
    FROM ranked_customers
    WHERE region_rank <= 5
    ORDER BY region, region_rank
""")

top_customers_query.show(40)
```

```
[Stage 73:=====> (33 + 1)
200]

[Stage 73:=====> (42 + 1)
200]

[Stage 73:=====> (54 + 1)
200]

[Stage 73:=====> (67 + 1)
200]

[Stage 73:=====> (80 + 1)
200]

[Stage 73:=====> (95 + 1)
200]

[Stage 73:=====> (106 + 1)
200]

[Stage 73:=====> (117 + 1)
200]

[Stage 73:=====> (125 + 1)
200]

[Stage 73:=====> (133 + 1)
200]

[Stage 73:=====> (141 + 1)
200]

[Stage 73:=====> (152 + 1)
200]
```

[Stage 73:=====> (165 + 1)
200]

[Stage 73:=====> (179 + 0)
200]

[Stage 73:=====> (189 + 1)
200]

region	customer_name	customer_revenue	region_rank
Alexandria	Amr Rashad	7924.6	1
Alexandria	Tarek Zaki	7555.1	2
Alexandria	Amr Kamal	5403.400000000001	3
Alexandria	Amr Hassan	5380.9	4
Alexandria	Fady Nasser	5175.75	5
Aswan	Hassan Adly	6556.4	1
Aswan	Tarek Fouad	5455.7	2
Aswan	Omar Zaki	5368.4	3
Aswan	Ziad Salem	4899.75	4
Aswan	Aya Gaber	4273.3	5
Cairo	Farida Naguib	8292.6	1
Cairo	Ali Zaki	5630.8	2
Cairo	Ziad Mansour	5620.5	3
Cairo	Yasmin Nasser	5545.5	4
Cairo	Laila Ibrahim	5383.4	5
Giza	Salma Nasser	6623.5	1
Giza	Amr Kamal	6492.5	2
Giza	Hassan Younis	5611.4	3
Giza	Mariam Ahmed	5263.65	4
Giza	Tarek Fouad	5148.0	5
Luxor	Sara Mansour	6588.75	1
Luxor	Yasmin Aziz	6253.349999999999	2
Luxor	Nour Farouk	5192.05	3
Luxor	Farida Zaki	5082.5	4
Luxor	Khaled Fahmy	4613.75	5
Mansoura	Omar El-Sayed	10978.1	1
Mansoura	Mariam Aziz	7003.1	2
Mansoura	Amr Aziz	6377.0	3
Mansoura	Khaled Zaki	6372.95	4
Mansoura	Amr Kamal	6217.2	5
Sharqia	Dina Salem	8824.4	1
Sharqia	Omar El-Sayed	5956.75	2
Sharqia	Heba Farouk	5815.15	3
Sharqia	Mona Gaber	5632.55	4
Sharqia	Hady Shawky	5603.85	5
Tanta	Amina Aziz	11669.4	1
Tanta	Aya Farouk	7027.2	2
Tanta	Heba Shawky	6890.6	3
Tanta	Tarek Zaki	6223.3	4
Tanta	Mariam Ibrahim	5934.1	5

. Window functions — running total & month-over-month growth

In [14]:

```
# Build a monthly category revenue table first (needed for the window examples below)
monthly_category = spark.sql("""
    SELECT category,
           date_format(order_date, 'yyyy-MM') AS order_month,
           SUM(line_total) AS monthly_revenue
    FROM silver_orders
    GROUP BY category, date_format(order_date, 'yyyy-MM')
""")

monthly_category.createOrReplaceTempView("monthly_category")
monthly_category.orderBy("category", "order_month").show(5)
```

```
[Stage 82:=====> (26 + 1)
200]

[Stage 82:=====> (35 + 1)
200]

[Stage 82:=====> (45 + 1)
200]

[Stage 82:=====> (60 + 1)
200]

[Stage 82:=====> (72 + 1)
200]

[Stage 82:=====> (86 + 1)
200]

[Stage 82:=====> (100 + 1)
200]

[Stage 82:=====> (116 + 1)
200]

[Stage 82:=====> (133 + 1)
200]

[Stage 82:=====> (147 + 1)
200]

[Stage 82:=====> (155 + 1)
200]

[Stage 82:=====> (164 + 1)
200]

[Stage 82:=====> (173 + 2)
200]

[Stage 82:=====> (186 + 1)
200]

[Stage 82:=====> (196 + 1)
200]
```

```
+-----+-----+-----+
|category|order_month|  monthly_revenue|
+-----+-----+-----+
|  Bakery|    2024-01|           7050.5|
|  Bakery|    2024-02|  8639.649999999998|
|  Bakery|    2024-03|10466.299999999997|
|  Bakery|    2024-04|           8897.25|
|  Bakery|    2024-05|           8879.6|
+-----+-----+-----+
only showing top 5 rows
```

In [15]:

```
# Window function example 1: running (cumulative) total per category, ordered by month
# ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW = "from the first row up to this one"
running_total_query = spark.sql("""
    SELECT category,
           order_month,
           monthly_revenue,
           SUM(monthly_revenue) OVER (
               PARTITION BY category
               ORDER BY order_month
               ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
           ) AS running_total
    FROM monthly_category
    ORDER BY category, order_month
""")

running_total_query.show(15)
```

```
[Stage 87:=====> (37 + 1)
200]

[Stage 87:=====> (51 + 1)
200]

[Stage 87:=====> (61 + 1)
200]

[Stage 87:=====> (75 + 1)
200]

[Stage 87:=====> (84 + 1)
200]

[Stage 87:=====> (99 + 1)
200]

[Stage 87:=====> (111 + 1)
200]

[Stage 87:=====> (121 + 1)
200]

[Stage 87:=====> (134 + 1)
200]

[Stage 87:=====> (148 + 1)
200]

[Stage 87:=====> (160 + 1)
200]

[Stage 87:=====> (175 + 1)
200]

[Stage 87:=====> (191 + 1)
200]
```

```

+-----+-----+-----+-----+
| category|order_month|  monthly_revenue|  running_total|
+-----+-----+-----+-----+
| Bakery| 2024-01|          7050.5|          7050.5|
| Bakery| 2024-02| 8639.649999999998| 15690.149999999998|
| Bakery| 2024-03|10466.299999999997| 26156.449999999997|
| Bakery| 2024-04|          8897.25|          35053.7|
| Bakery| 2024-05|          8879.6| 43933.299999999996|
| Bakery| 2024-06|10168.999999999996| 54102.299999999999|
| Bakery| 2024-07|10665.250000000004| 64767.549999999999|
| Bakery| 2024-08| 9019.399999999998| 73786.949999999998|
| Bakery| 2024-09|         10174.5| 83961.449999999998|
| Bakery| 2024-10| 9180.049999999997| 93141.499999999999|
| Bakery| 2024-11|          7023.95|100165.449999999998|
| Bakery| 2024-12|          8680.35|108845.799999999999|
| Beverages| 2024-01|15514.150000000001|15514.150000000001|
| Beverages| 2024-02|         16982.95|32497.100000000002|
| Beverages| 2024-03|         17329.0|49826.100000000006|
+-----+-----+-----+-----+

```

only showing top 15 rows

In [16]:

```
# Window function example 2: month-over-month growth using LAG()
# LAG(column, 1) fetches the value from the *previous* row in the same partition
mom_growth_query = spark.sql("""
    SELECT category,
           order_month,
           monthly_revenue,
           LAG(monthly_revenue, 1) OVER (
               PARTITION BY category ORDER BY order_month
           ) AS prev_month_revenue,
           ROUND(
               (monthly_revenue - LAG(monthly_revenue, 1) OVER (PARTITION BY category ORDER BY
                   / LAG(monthly_revenue, 1) OVER (PARTITION BY category ORDER BY order_month) * 10
               2) AS mom_growth_pct
    FROM monthly_category
    ORDER BY category, order_month
""")

mom_growth_query.show(15)
```

```
[Stage 96:====> (18 + 1)
200]

[Stage 96:=====> (26 + 1)
200]

[Stage 96:=====> (34 + 1)
200]

[Stage 96:=====> (46 + 0)
200]

[Stage 96:=====> (59 + 1)
200]

[Stage 96:=====> (71 + 1)
200]

[Stage 96:=====> (84 + 2)
200]

[Stage 96:=====> (98 + 1)
200]

[Stage 96:=====> (112 + 1)
200]

[Stage 96:=====> (128 + 1)
200]

[Stage 96:=====> (142 + 1)
200]

[Stage 96:=====> (150 + 1)
200]

[Stage 96:=====> (165 + 1)
200]
```


[Stage 96:=====> (177 + 2)
200]

[Stage 96:=====> (185 + 1)
200]

[Stage 96:=====> (193 + 1)
200]

category	order_month	monthly_revenue	prev_month_revenue	mom_growth_p
Bakery	2024-01	7050.5	NULL	NU
Bakery	2024-02	8639.649999999998	7050.5	22.
Bakery	2024-03	10466.299999999997	8639.649999999998	21.
Bakery	2024-04	8897.25	10466.299999999997	-14.
Bakery	2024-05	8879.6	8897.25	-0
Bakery	2024-06	10168.999999999996	8879.6	14.
Bakery	2024-07	10665.250000000004	10168.999999999996	4.
Bakery	2024-08	9019.399999999998	10665.250000000004	-15.
Bakery	2024-09	10174.5	9019.399999999998	12.
Bakery	2024-10	9180.049999999997	10174.5	-9.
Bakery	2024-11	7023.95	9180.049999999997	-23.
Bakery	2024-12	8680.35	7023.95	23.
Beverages	2024-01	15514.150000000001	NULL	NU
Beverages	2024-02	16982.95	15514.150000000001	9.
Beverages	2024-03	17329.0	16982.95	2.

only showing top 15 rows

In [17]:

```
# Window function example 3 (DataFrame API): best-selling product per category
category_window = Window.partitionBy("category").orderBy(F.col("product_revenue").desc())

# First aggregate revenue per product
product_revenue = (
    silver_orders.groupBy("category", "product_name")
    .agg(F.sum("line_total").alias("product_revenue"))
)

# Then rank products inside each category using the window spec
best_sellers = (
    product_revenue
    .withColumn("rank_in_category", F.rank().over(category_window))
    .filter(F.col("rank_in_category") == 1)
    .orderBy("category")
)

best_sellers.show()
```

```
[Stage 105:=====> (44 + 1)
200]

[Stage 105:=====> (59 + 1)
200]

[Stage 105:=====> (72 + 1)
200]

[Stage 105:=====> (82 + 1)
200]

[Stage 105:=====> (96 + 1)
200]

[Stage 105:=====> (113 + 1)
200]

[Stage 105:=====> (130 + 1)
200]

[Stage 105:=====> (144 + 0)
200]

[Stage 105:=====> (159 + 1)
200]

[Stage 105:=====> (175 + 1)
200]

[Stage 105:=====> (190 + 2)
200]
```

category	product_name	product_revenue	rank_in_category
Bakery	Cheesecake Slice	31050.5	1
Beverages	Cappuccino	26688.75	1
Merchandise	Coffee Bean Bag 1kg	324187.5	1
Snacks	Club Sandwich	42531.5	1

. Save the gold tables

Write the finished reports out as CSV files, ready to hand off to a BI tool or another notebook

In [18]:

```
# coalesce(1) merges all partitions into a single output file (fine at this data size)
# mode("overwrite") replaces any existing output from a previous run
category_report.coalesce(1).write.mode("overwrite").option("header", True).csv("gold_category_r
monthly_trend.coalesce(1).write.mode("overwrite").option("header", True).csv("gold_monthly_tren
best_sellers.coalesce(1).write.mode("overwrite").option("header", True).csv("gold_best_sellers"

print("Gold reports written to disk.")
```

```
[Stage 114:=====> (19 + 1)
200]

[Stage 114:=====> (31 + 1)
200]

[Stage 114:=====> (40 + 1)
200]

[Stage 114:=====> (48 + 1)
200]

[Stage 114:=====> (56 + 1)
200]

[Stage 114:=====> (68 + 1)
200]

[Stage 114:=====> (80 + 1)
200]

[Stage 114:=====> (89 + 1)
200]

[Stage 114:=====> (101 + 1)
200]

[Stage 114:=====> (114 + 1)
200]

[Stage 114:=====> (124 + 1)
200]

[Stage 114:=====> (134 + 1)
200]

[Stage 114:=====> (147 + 1)
200]

[Stage 114:=====> (159 + 1)
200]

[Stage 114:=====> (171 + 1)
200]

[Stage 114:=====> (184 + 1)
200]

[Stage 114:=====> (197 + 1)
200]
```

[Stage 132:=====> 200]	(23 + 1)
[Stage 132:=====> 200]	(32 + 1)
[Stage 132:=====> 200]	(42 + 1)
[Stage 132:=====> 200]	(50 + 1)
[Stage 132:=====> 200]	(64 + 1)
[Stage 132:=====> 200]	(80 + 1)
[Stage 132:=====> 200]	(96 + 1)
[Stage 132:=====> 200]	(113 + 1)
[Stage 132:=====> 200]	(129 + 1)
[Stage 132:=====> 200]	(147 + 1)
[Stage 132:=====> 200]	(158 + 1)
[Stage 132:=====> 200]	(174 + 1)
[Stage 132:=====> 200]	(186 + 1)
[Stage 144:=====> 200]	(41 + 1)
[Stage 144:=====> 200]	(52 + 1)
[Stage 144:=====> 200]	(68 + 1)
[Stage 144:=====> 200]	(81 + 1)
[Stage 144:=====> 200]	(96 + 1)
[Stage 144:=====> 200]	(111 + 1)
[Stage 144:=====> 200]	(126 + 1)

```
[Stage 144:=====> (142 + 1)
200]

[Stage 144:=====> (156 + 1)
200]

[Stage 144:=====> (171 + 1)
200]

[Stage 144:=====> (187 + 1)
200]
```

Gold reports written to disk.

. Wrap-up — stop the Spark session

In [19]:

```
# Release the cached table from memory
silver_orders.unpersist()

# Always stop the SparkSession when finished, to free local resources
spark.stop()

print("Spark session stopped. Project complete!")
```

Spark session stopped. Project complete!

What this project practiced

- Structuring work into **bronze** → **silver** → **gold** layers, like a real data pipeline
- Discovering data quality issues (duplicates, nulls, invalid values) before trusting data
- Cleaning data with `dropDuplicates`, `dropna`, `filter`
- **Joining** multiple tables with `.join(..., on=..., how=...)`
- `groupBy` + `agg` for business reports
- SparkSQL **CTEs** (`WITH ... AS (...)`) for multi-step ranked queries
- **Window functions**: running totals (`ROWS BETWEEN ...`), month-over-month growth (`LAG()`), and ranking with `RANK()` in both SQL and the DataFrame API
- Writing finished "gold" tables back out to disk

Suggested next steps

- Add a `ROW_NUMBER()` version of the top-customers query and compare it to `RANK()`
- Try `LEAD()` to look one month *forward* instead of back
- Swap the CSV writes for `.parquet(...)` and compare file size / read speed